20CYS304 – ARTIFICIAL INTELLIGENCE AND NEURAL NETWORKS

MITREMapper – Project Report

Deepak Sasidharan – CB.SC.U4CYS23010

Dinesh V Masuti – CB.SC.U4CYS23012

Karun A.T. – CB.SC.U4CYS23023

Rohit Sundar V – CB.SC.U4CYS23039

Problem Statement

- Cyber Threat Intelligence (CTI) reports are initially written (or are at least thought of) based on observed behaviours in natural language by threat analysts.
- These reports, written in natural language, have to be mapped to the MITRE ATT&CK Tactics-Techniques-Procedures framework, which can be difficult and laborious to do manually.

Proposed Solution

In today's digital world, cyber threats aren't just something that affects organizations or governments — they're personal. Every day, individuals fall victim to phishing scams, malware infections, or social engineering attacks. However, most people don't know how or why it happened — and more importantly, what exactly the hacker did.

To bridge this gap, we developed MITREMapper — a system that not only helps cybersecurity professionals by classifying CTI reports and mapping them to MITRE ATT&CK techniques, but also introduces a novel feature: personalized threat mapping for individuals.

Implementation Details

Scope

Input: Free text in natural language.

Outputs:

- The problem is a hierarchical multi-label classification problem.
- Structured Mapping
  - {tactic, technique, sub-technique, procedure, threat-actors, text, confidence}
- Action Plan
  - Organizations: Mitigations from MITRE ATT&CK framework (with ATT&CK IDs).
  - Individuals: Immediate to-do checklist.
- Information: Description of what happened and how it happened.

Screenshots of Results and Graphs

regularly review and update their security measures. The attackers' ability to move laterally across the network and exfiltrate sensitive data underscores the importance of implementing robust security controls, such as multi-factor authentication and regular security audits, to prevent similar attacks in the future.

🎯 MITRE ATT&CK Mappings

Initial Access — 90% Confidence

T1218 - System Binary Proxy Execution

└ T1218.007 - Msiexec

Procedure: Spear-phishing

Evidence: *"The threat actor APT29 compromised the organization's network through a spear-phishing email containing a malicious attachment."*

Threat Actors: APT29

Persistence — 80% Confidence

T1053 - Scheduled Task/Job

└ T1053.005 - Scheduled Task

Procedure: Abused the Windows Task Scheduler to perform task scheduling for initial or recurring execution of malicious code

Evidence: *"establishing persistence using scheduled tasks"*

Threat Actors: APT29

Lateral Movement — 90% Confidence

T1003 - OS Credential Dumping

└ T1003.001 - LSASS Memory

Procedure: Used stolen credentials to move laterally across the network

Evidence: *"lateral movement using stolen credentials"*

Threat Actors: APT29

T1053 - Scheduled Task/Job

└ T1053.005 - Scheduled Task

Procedure: Abused the Windows Task Scheduler to perform task scheduling for initial or recurring execution of malicious code

Evidence: *"establishing persistence using scheduled tasks"*

Threat Actors: APT29

Lateral Movement — 90% Confidence

T1003 - OS Credential Dumping

└ T1003.001 - LSASS Memory

Procedure: Used stolen credentials to move laterally across the network

Evidence: *"lateral movement using stolen credentials"*

Threat Actors: APT29

Exfiltration — 95% Confidence

T1070 - Indicator Removal

└ T1070.002 - Clear Linux or Mac System Logs

Procedure: Used PowerShell scripts to exfiltrate sensitive data to a command-and-control server

Evidence: *"deploying PowerShell scripts for data exfiltration"*

Threat Actors: APT29

Execution — 90% Confidence

T1047 - Windows Management Instrumentation

Procedure: Abused Windows Management Instrumentation (WMI) to execute malicious commands and payloads

Evidence: *"using WMI for remote execution"*

Threat Actors: APT29

For Organizations

[M1050] Exploit Protection

Deploy capabilities that detect, block, and mitigate conditions indicative of software exploits. These capabilities aim to prevent exploitation by addressing vulnerabilities, monitoring anomalous beha...

[M1037] Filter Network Traffic

Employ network appliances and endpoint software to filter ingress, egress, and lateral network traffic. This includes protocol-based filtering, enforcing firewall rules, and blocking or restricting tr...

[M1026] Privileged Account Management

Privileged Account Management focuses on implementing policies, controls, and tools to securely manage privileged accounts (e.g., SYSTEM, root, or administrative accounts). This includes restricting a...

[M1038] Execution Prevention

Prevent the execution of unauthorized or malicious code on systems by implementing application control, script blocking, and other execution prevention mechanisms. This ensures that only trusted and a...

[M1042] Disable or Remove Feature or Program

Disable or remove unnecessary and potentially vulnerable software, features, or services to reduce the attack surface and prevent abuse by adversaries. This involves identifying software or features t...

[M1021] Restrict Web-Based Content

Restricting web-based content involves enforcing policies and technologies that limit access to potentially malicious websites, unsafe downloads, and unauthorized browser behaviors. This can include U...

[M1018] User Account Management

User Account Management involves implementing and enforcing policies for the lifecycle of user accounts, including creation, modification, and deactivation. Proper account management reduces the attac...

[M1021] Restrict Web-Based Content

Restricting web-based content involves enforcing policies and technologies that limit access to potentially malicious websites, unsafe downloads, and unauthorized browser behaviors. This can include U...

[M1018] User Account Management

User Account Management involves implementing and enforcing policies for the lifecycle of user accounts, including creation, modification, and deactivation. Proper account management reduces the attac...

[M1028] Operating System Configuration

Operating System Configuration involves adjusting system settings and hardening the default configurations of an operating system (OS) to mitigate adversary exploitation and prevent abuse of system fu...

[M1022] Restrict File and Directory Permissions

Restricting file and directory permissions involves setting access controls at the file system level to limit which users, groups, or processes can read, write, or execute files. By configuring permis...

[M1047] Audit

Auditing is the process of recording activity and systematically reviewing and analyzing the activity and system configurations. The primary purpose of auditing is to detect anomalies and identify pot...

[M1041] Encrypt Sensitive Information

Protect sensitive information at rest, in transit, and during processing by using strong encryption algorithms. Encryption ensures the confidentiality and integrity of data, preventing unauthorized ac...

[M1040] Behavior Prevention on Endpoint

Behavior Prevention on Endpoint refers to the use of technologies and strategies to detect and block potentially malicious activities by analyzing the behavior of processes, files, API calls, and othe...

[M1027] Password Policies

Set and enforce secure password policies for accounts to reduce the likelihood of unauthorized access. Strong password policies include enforcing password complexity, requiring regular password change...

[M1017] User Training

User Training involves educating employees and contractors on recognizing, reporting, and preventing cyber threats that rely on human interaction, such as phishing, social engineering, and other manip...

[M1025] Privileged Process Integrity

Privileged Process Integrity focuses on defending highly privileged processes (e.g., system services, antivirus, or authentication processes) from tampering, injection, or compromise by adversaries. T...

[M1043] Credential Access Protection

Credential Access Protection focuses on implementing measures to prevent adversaries from obtaining credentials, such as passwords, hashes, tokens, or keys, that could be used for unauthorized access....

[M1015] Active Directory Configuration

Implement robust Active Directory (AD) configurations using group policies to secure user accounts, control access, and minimize the attack surface. AD configurations enable centralized control over a...

[M1029] Remote Data Storage

Remote Data Storage focuses on moving critical data, such as security logs and sensitive files, to secure, off-host locations to minimize unauthorized access, tampering, or destruction by adversaries....

Immediate Actions for Individuals

1. Immediately disconnect the affected system from the network
2. Change all passwords from a clean device
3. Enable multi-factor authentication on all accounts
4. Run a full system antivirus scan
5. Review recent account activity for unauthorized access
6. Back up important files to secure offline storage
7. Contact your IT department or security team
8. Monitor financial accounts for suspicious activity

```
 * Debugger is active!
 * Debugger PIN: 126-446-563
127.0.0.1 - - [12/Oct/2025 22:22:42] "GET / HTTP/1.1" 200 -
127.0.0.1 - - [12/Oct/2025 22:22:42] "GET / HTTP/1.1" 304 -
127.0.0.1 - - [12/Oct/2025 22:22:43] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [12/Oct/2025 22:23:24] "OPTIONS /api/analyze HTTP/1.1" 200 -

============================================================
Analyzing threat report...
============================================================


=== Starting Threat Analysis ===

Step 1: Extracting threat indicators...
Found: 1 threat actors, 5 behaviors

Step 2: Mapping to MITRE ATT&CK TTPs...
Identified 5 TTP mappings

Step 3: Generating remediation action plan...
Created action plan with 18 org mitigations and 8 individual actions

Step 4: Generating threat summary...

=== Analysis Complete ===


Analysis complete! Found 5 TTP mappings
127.0.0.1 - - [12/Oct/2025 22:24:49] "POST /api/analyze HTTP/1.1" 200 -
```

Explanations:

CELL 1 :

This cell imports all the necessary Python libraries for the project to function.

- json, re, typing, dataclasses, pathlib: These are standard Python libraries for handling data. json is for working with JSON data (like the MITRE ATT&CK matrices) , re is for regular expressions (used to extract JSON from the model's text output) , typing helps define expected data types for cleaner code , dataclasses provide an easy way to create classes for storing data, and pathlib helps manage file paths.

- torch: An open-source machine learning library that provides the underlying tensor computation and deep learning functionalities required by the transformers library.

- transformers: A library by Hugging Face that provides pre-trained models like Llama 3.2. This is used to load the model (AutoModelForCausalLM) and its corresponding tokenizer (AutoTokenizer).

- sentence_transformers: This library is used to generate high-quality embeddings (numerical representations) of text. It's used here to create embeddings for MITRE techniques to find the most relevant ones for a given report.

- numpy: A fundamental package for scientific computing in Python, used here for efficient numerical operations on the embeddings.

- sklearn.metrics.pairwise.cosine_similarity: A function from the Scikit-learn library to calculate the cosine similarity between the threat report's embedding and the technique embeddings, which helps find the most relevant techniques

CELL 2 :

This cell defines the custom data structures used to organize the project's inputs and outputs neatly. Using @dataclass decorator makes these classes concise and easy to read.

- TTMapping: Represents a single mapping of a text snippet from a threat report to a specific MITRE ATT&CK Tactic, Technique, and Procedure (TTP). It stores the tactic, technique name, optional sub-technique, the specific procedure observed, threat actors involved, the original text evidence, and a confidence score for the mapping.

- ActionPlan: This class holds the generated remediation advice. It is split into two parts: a list of structured mitigations for organizations (including mitigation name, ATT&CK ID, and description) and a simple checklist of immediate actions for individuals.

- ThreatAnalysis: This is the main output class that consolidates all the results. It contains a list of all TTMapping objects found , the complete ActionPlan , and a human-readable summary of the incident. The to_dict methods in these classes are used to easily convert the results into a format that can be saved as a JSON file.

CELL 3 :

The MITREKnowledgeBase class is responsible for loading, parsing, and managing all the data from the MITRE ATT&CK framework files. It acts as a local database for the application.

- __init__: The constructor loads the enterprise, ICS, and mobile ATT&CK matrices from local JSON files. It also loads a file of historical mappings, which can be used for few-shot examples.

- _build_..._lookup Methods: These methods process the raw MITRE data to create fast and convenient lookup tables (dictionaries).

- _build_technique_lookup: Creates a dictionary where the key is a technique ID (e.g., "T1053") and the value contains its full details like name, description, and associated tactics.

- _build_tactic_lookup: Does the same for tactics (e.g., "TA0003" maps to "Persistence").

- _build_mitigation_lookup: Builds a mapping from a technique ID to a list of possible mitigation strategies.

- get_similar_mappings: This function retrieves historical mapping examples that are similar to the input text. These examples are used in the prompt to show the Large Language Model (LLM) exactly how to perform the mapping task (a technique called "few-shot learning").

CELL 4 :

This initializes the core class of the project where the main analysis logic resides. It integrates the knowledge base and the LLM to perform the threat analysis.

- __init__: Initializes the mapper. It loads the pre-quantized LLM (unsloth/Llama-3.2-3B-Instruct-bnb-4bit) and tokenizer from Hugging Face. A quantized model is used to reduce memory usage, allowing it to run on less powerful hardware. It also loads the SentenceTransformer model for creating embeddings.

- Embedding and Semantic Search:
    - _create_technique_embeddings: This method runs once at startup to convert all MITRE technique descriptions into numerical vectors (embeddings).

- _find_relevant_techniques: When a new report is analyzed, this function converts the report text into an embedding and uses cosine similarity to find the most semantically similar techniques from the knowledge base. This is a form of Retrieval-Augmented Generation (RAG), which provides the LLM with relevant context to improve its accuracy.

- LLM Interaction and Analysis Pipeline:
    - _generate_with_llm: A helper function that formats a prompt, sends it to the Llama model, and returns the generated text response.

- _extract_threat_indicators: The first step of the analysis, where the LLM is asked to pull out structured data like threat actors and malicious behaviors from the raw text.

- _map_to_ttps: The main mapping function. It constructs a detailed prompt for the LLM that includes few-shot examples (from get_similar_mappings) and relevant techniques (from _find_relevant_techniques). It then asks the LLM to output a JSON array of TTP mappings, which are then parsed into TTMapping objects.

- _generate_action_plan: After identifying the TTPs, this function generates remediation steps. It retrieves mitigations for organizations from the knowledge base and uses the LLM to generate a custom, prioritized checklist for individuals.

- _generate_summary: Uses the LLM one last time to create a simple, non-technical summary of the attack, explaining what happened, how, and why it matters.

- analyze: This is the main public method that executes the entire pipeline in order: extract indicators -> map to TTPs -> generate action plan -> generate summary

CELL 5 :

This final cell demonstrates how to use the MITREMapper class and serves as the entry point when the script is run directly.

- It initializes the MITREKnowledgeBase and the MITREMapper.

- An example cyber threat intelligence report about the actor APT29 is defined.

- The mapper.analyze() method is called with this report to start the analysis.

- Finally, it prints the complete, formatted results—summary, TTP mappings, and action plans for both organizations and individuals—to the console and saves the structured output to a JSON file named threat_analysis_results.json.

Project Code

app.ipynb:

```
# Cell 1
import json
import re
from typing import List, Dict, Any, Optional, Tuple
from dataclasses import dataclass, asdict
from pathlib import Path
import torch
from transformers import AutoTokenizer, AutoModelForCausalLM
from sentence_transformers import SentenceTransformer
import numpy as np
from sklearn.metrics.pairwise import cosine_similarity
```

```
# Cell 2
@dataclass
class TTMapping:
    """Structured mapping output"""
    tactic: str
    technique: str
    sub_technique: Optional[str]
    procedure: str
    threat_actors: List[str]
    text: str  # Original text snippet from report
    confidence: float

    def to_dict(self):
        return asdict(self)

@dataclass
class ActionPlan:
    """Remediation action plan"""
    for_organizations: List[Dict[str, str]]  # {mitigation, attack_id, description}
    for_individuals: List[str]  # Immediate checklist items

@dataclass
class ThreatAnalysis:
    """Complete analysis output"""
    mappings: List[TTMapping]
    action_plan: ActionPlan
    summary: str  # Description of what happened and how

    def to_dict(self):
        return {
```

```
        "mappings": [m.to_dict() for m in self.mappings],
        "action_plan": {
            "for_organizations": self.action_plan.for_organizations,
            "for_individuals": self.action_plan.for_individuals
        },
        "summary": self.summary
    }

# Cell 3
class MITREKnowledgeBase:
    """Manages MITRE ATT&CK matrices and mappings"""

    def __init__(
        self,
        enterprise_path: str = "attack-matrices/enterprise-attack.json",
        ics_path: str = "attack-matrices/ics-attack.json",
        mobile_path: str = "attack-matrices/mobile-attack.json",
        mappings_path: str = "mitre_mappings.json"
    ):
        self.enterprise = self._load_matrix(enterprise_path)
        self.ics = self._load_matrix(ics_path)
        self.mobile = self._load_matrix(mobile_path)
        self.historical_mappings = self._load_mappings(mappings_path)

        # Build lookup tables
        self.technique_lookup = self._build_technique_lookup()
        self.tactic_lookup = self._build_tactic_lookup()
        self.mitigation_lookup = self._build_mitigation_lookup()

    def _load_matrix(self, path: str) -> Dict:
```

```
    """Load MITRE ATT&CK matrix"""
    try:
        with open(path, 'r', encoding='utf-8') as f:
            return json.load(f)
    except FileNotFoundError:
        print(f"Warning: {path} not found")
        return {"objects": []}

def _load_mappings(self, path: str) -> List[Dict]:
    """Load historical mappings for few-shot examples"""
    try:
        with open(path, 'r', encoding='utf-8') as f:
            return json.load(f)
    except FileNotFoundError:
        print(f"Warning: {path} not found")
        return []

def _build_technique_lookup(self) -> Dict[str, Dict]:
    """Build technique ID -> full details lookup"""
    lookup = {}
    for matrix in [self.enterprise, self.ics, self.mobile]:
        for obj in matrix.get("objects", []):
            if obj.get("type") == "attack-pattern":
                tech_id = None
                for ref in obj.get("external_references", []):
                    if ref.get("source_name") == "mitre-attack":
                        tech_id = ref.get("external_id")
                        break

                if tech_id:
```

```
                lookup[tech_id] = {
                    "id": tech_id,
                    "name": obj.get("name"),
                    "description": obj.get("description", ""),
                    "platforms": obj.get("x_mitre_platforms", []),
                    "tactics": [phase["phase_name"] for phase in obj.get("kill_chain_phases",
[])],
                    "is_subtechnique": "." in tech_id
                }
        return lookup

    def _build_tactic_lookup(self) -> Dict[str, Dict]:
        """Build tactic ID -> details lookup"""
        lookup = {}
        for matrix in [self.enterprise, self.ics, self.mobile]:
            for obj in matrix.get("objects", []):
                if obj.get("type") == "x-mitre-tactic":
                    tactic_id = None
                    for ref in obj.get("external_references", []):
                        if ref.get("source_name") == "mitre-attack":
                            tactic_id = ref.get("external_id")
                            break

                    if tactic_id:
                        lookup[tactic_id] = {
                            "id": tactic_id,
                            "name": obj.get("name"),
                            "description": obj.get("description", "")
                        }
        return lookup
```

```
def _build_mitigation_lookup(self) -> Dict[str, List[Dict]]:
    """Build technique ID -> mitigations lookup"""
    # First, get all mitigations
    mitigations = {}
    for matrix in [self.enterprise, self.ics, self.mobile]:
        for obj in matrix.get("objects", []):
            if obj.get("type") == "course-of-action":
                mit_id = None
                for ref in obj.get("external_references", []):
                    if ref.get("source_name") == "mitre-attack":
                        mit_id = ref.get("external_id")
                        break

                if mit_id:
                    mitigations[obj["id"]] = {
                        "id": mit_id,
                        "name": obj.get("name"),
                        "description": obj.get("description", "")
                    }

    # Build technique -> mitigations mapping
    lookup = {}
    for matrix in [self.enterprise, self.ics, self.mobile]:
        for obj in matrix.get("objects", []):
            if obj.get("type") == "relationship" and obj.get("relationship_type") == "mitigates":
                # Find the technique ID
                target_id = obj.get("target_ref")
                source_id = obj.get("source_ref")

                if source_id in mitigations:
```

```
                # Find technique external ID
                for tech_obj in matrix.get("objects", []):
                    if tech_obj.get("id") == target_id and tech_obj.get("type") ==
"attack-pattern":
                        tech_ext_id = None
                        for ref in tech_obj.get("external_references", []):
                            if ref.get("source_name") == "mitre-attack":
                                tech_ext_id = ref.get("external_id")
                                break

                        if tech_ext_id:
                            if tech_ext_id not in lookup:
                                lookup[tech_ext_id] = []
                            lookup[tech_ext_id].append(mitigations[source_id])

        return lookup

    def get_similar_mappings(self, text: str, top_k: int = 3) -> List[Dict]:
        """Retrieve similar historical mappings for few-shot examples"""
        if not self.historical_mappings:
            return []

        # Simple keyword-based similarity (can be improved with embeddings)
        text_lower = text.lower()
        scored_mappings = []

        for mapping in self.historical_mappings:
            mapping_text = mapping.get("text", "").lower()
            # Simple word overlap score
            text_words = set(text_lower.split())
            mapping_words = set(mapping_text.split())
```

```
            overlap = len(text_words & mapping_words)

            if overlap > 0:
                scored_mappings.append((overlap, mapping))

        scored_mappings.sort(reverse=True, key=lambda x: x[0])
        return [m[1] for m in scored_mappings[:top_k]]

# Cell 4
class MITREMapper:
    def __init__(
        self,
        model_name: str = "unsloth/Llama-3.2-3B-Instruct-bnb-4bit",  # Changed default
        knowledge_base: Optional[MITREKnowledgeBase] = None
    ):
        """
        Initialize MITREMapper with Llama model and MITRE knowledge base.

        Args:
            model_name: HuggingFace model identifier
            knowledge_base: Pre-loaded MITRE knowledge base
            use_4bit: Use 4-bit quantization to fit in 6GB VRAM
        """
        self.knowledge_base = knowledge_base or MITREKnowledgeBase()

        quantized_model_name = "unsloth/Llama-3.2-3B-Instruct-bnb-4bit"

        print(f"Loading pre-quantized model: {quantized_model_name}...")
        self.tokenizer = AutoTokenizer.from_pretrained(quantized_model_name)
```

```
    self.model = AutoModelForCausalLM.from_pretrained(
        quantized_model_name,
        device_map="auto",
        low_cpu_mem_usage=True,
        trust_remote_code=True  # Required for some quantized models
    )

    # Load sentence transformer for semantic similarity (lighter model)
    print("Loading sentence transformer for embeddings...")
    self.embedder = SentenceTransformer('all-MiniLM-L6-v2')

    # Create technique embeddings for semantic search
    self._create_technique_embeddings()

def _create_technique_embeddings(self):
    """Create embeddings for all MITRE techniques"""
    print("Creating technique embeddings...")
    self.technique_texts = []
    self.technique_ids = []

    for tech_id, tech_data in self.knowledge_base.technique_lookup.items():
        text = f"{tech_data['name']}: {tech_data['description'][:200]}"
        self.technique_texts.append(text)
        self.technique_ids.append(tech_id)

    self.technique_embeddings = self.embedder.encode(
        self.technique_texts,
        show_progress_bar=True
    )
```

```
    def _find_relevant_techniques(self, text: str, top_k: int = 10) -> List[Tuple[str, float]]:
        """Find most relevant techniques using semantic similarity"""
        query_embedding = self.embedder.encode([text])
        similarities = cosine_similarity(query_embedding, self.technique_embeddings)[0]

        top_indices = np.argsort(similarities)[-top_k:][::-1]
        return [(self.technique_ids[i], float(similarities[i])) for i in top_indices]

    def _generate_with_llm(self, prompt: str, max_tokens: int = 2048) -> str:
        """Generate response from Llama model"""
        messages = [
            {"role": "system", "content": "You are a cybersecurity expert specializing in MITRE
ATT&CK framework mapping and threat analysis."},
            {"role": "user", "content": prompt}
        ]

        input_text = self.tokenizer.apply_chat_template(
            messages,
            tokenize=False,
            add_generation_prompt=True
        )

        inputs = self.tokenizer(input_text, return_tensors="pt").to(self.model.device)

        with torch.no_grad():
            outputs = self.model.generate(
                **inputs,
                max_new_tokens=max_tokens,
                temperature=0.7,
                top_p=0.9,
                do_sample=True,
```

```
            pad_token_id=self.tokenizer.eos_token_id
        )

    response = self.tokenizer.decode(outputs[0][inputs['input_ids'].shape[1]:],
skip_special_tokens=True)
    return response.strip()

  def _extract_threat_indicators(self, report: str) -> Dict[str, Any]:
    """Extract key indicators from threat report"""
    prompt = f"""Analyze this threat intelligence report and extract key information:

REPORT:
{report}

Extract and provide in JSON format:
1. threat_actors: List of threat actor names/groups mentioned (if any)
2. key_behaviors: List of specific malicious behaviors described
3. indicators: Technical indicators (IPs, domains, file hashes, commands, etc.)
4. platforms: Affected platforms (Windows, Linux, macOS, etc.)

Respond ONLY with valid JSON."""

    response = self._generate_with_llm(prompt, max_tokens=1024)

    try:
        # Extract JSON from response
        json_match = re.search(r'\{.*\}', response, re.DOTALL)
        if json_match:
            return json.loads(json_match.group())
    except json.JSONDecodeError:
        pass
```

```
        # Fallback to basic extraction
        return {
            "threat_actors": [],
            "key_behaviors": [report[:200]],
            "indicators": [],
            "platforms": []
        }

    def _map_to_ttps(self, report: str, extracted_info: Dict) -> List[TTMapping]:
        """Map report to MITRE ATT&CK TTPs using hierarchical classification"""

        # Get similar historical mappings for few-shot learning
        similar_mappings = self.knowledge_base.get_similar_mappings(report, top_k=3)

        # Get relevant techniques using semantic search
        relevant_techniques = self._find_relevant_techniques(report, top_k=15)

        # Build context with technique details
        technique_context = "\n\n".join([
            f"ID: {tech_id}\nName:
{self.knowledge_base.technique_lookup[tech_id]['name']}\n"
            f"Description:
{self.knowledge_base.technique_lookup[tech_id]['description'][:150]}...\n"
            f"Tactics: {', '.join(self.knowledge_base.technique_lookup[tech_id]['tactics'])}"
            for tech_id, _ in relevant_techniques[:10]
        ])

        # Build few-shot examples
        few_shot_examples = ""
        if similar_mappings:
```

```
        few_shot_examples = "Here are examples of similar threat reports mapped to MITRE ATT&CK:\n\n"
        for i, mapping in enumerate(similar_mappings[:2], 1):
            few_shot_examples += f"Example {i}:\n"
            few_shot_examples += f"Text: {mapping.get('text', '')}\n"
            few_shot_examples += f"Technique: {mapping['technique']['id']} - {mapping['technique']['name']}\n"
            if mapping.get('sub-technique'):
                few_shot_examples += f"Sub-technique: {mapping['sub-technique']['id']} - {mapping['sub-technique']['name']}\n"
            few_shot_examples += f"Tactic: {mapping['tactic']['name']}\n\n"

    prompt = f"""{few_shot_examples}

RELEVANT MITRE ATT&CK TECHNIQUES:
{technique_context}

THREAT REPORT TO ANALYZE:
{report}

EXTRACTED INFORMATION:
- Threat Actors: {', '.join(extracted_info.get('threat_actors', [])) or 'Unknown'}
- Key Behaviors: {'; '.join(extracted_info.get('key_behaviors', []))}
- Platforms: {', '.join(extracted_info.get('platforms', [])) or 'Unknown'}

TASK:
Map this threat report to MITRE ATT&CK framework. For EACH distinct malicious behavior or technique mentioned:

1. Identify the specific text snippet describing the behavior
2. Map to the most appropriate Technique ID from the relevant techniques above
3. If applicable, identify the Sub-technique ID
```

```
4. Identify the Tactic (e.g., Initial Access, Persistence, etc.)
5. Describe the specific procedure used
6. Assign a confidence score (0.0-1.0)

Respond with a JSON array of mappings in this EXACT format:
[
  {{
    "text": "exact quote from report",
    "technique_id": "T1059",
    "sub_technique_id": "T1059.001",
    "tactic": "Execution",
    "procedure": "description of how technique was used",
    "confidence": 0.95
  }}
]

Respond ONLY with the JSON array, no other text."""

        response = self._generate_with_llm(prompt, max_tokens=2048)

        # Parse response
        try:
            json_match = re.search(r'\[.*\]', response, re.DOTALL)
            if json_match:
                mappings_data = json.loads(json_match.group())

                # Convert to TTMapping objects
                tt_mappings = []
                for mapping in mappings_data:
                    tech_id = mapping.get('technique_id', '')
```

```
            sub_tech_id = mapping.get('sub_technique_id')

            # Get technique details
            tech_info = self.knowledge_base.technique_lookup.get(tech_id, {})
            tech_name = f"{tech_id} - {tech_info.get('name', 'Unknown')}"

            sub_tech_name = None
            if sub_tech_id:
                sub_tech_info = self.knowledge_base.technique_lookup.get(sub_tech_id, {})
                sub_tech_name = f"{sub_tech_id} - {sub_tech_info.get('name', 'Unknown')}"

            tt_mappings.append(TTMapping(
                tactic=mapping.get('tactic', 'Unknown'),
                technique=tech_name,
                sub_technique=sub_tech_name,
                procedure=mapping.get('procedure', ''),
                threat_actors=extracted_info.get('threat_actors', []),
                text=mapping.get('text', ''),
                confidence=mapping.get('confidence', 0.5)
            ))

        return tt_mappings
    except (json.JSONDecodeError, KeyError) as e:
        print(f"Error parsing LLM response: {e}")

    # Fallback: create basic mapping from top relevant technique
    if relevant_techniques:
        top_tech_id, confidence = relevant_techniques[0]
        tech_info = self.knowledge_base.technique_lookup[top_tech_id]
```

```
        return [TTMapping(
            tactic=tech_info['tactics'][0] if tech_info['tactics'] else 'Unknown',
            technique=f"{top_tech_id} - {tech_info['name']}",
            sub_technique=None,
            procedure="Automated mapping based on semantic similarity",
            threat_actors=extracted_info.get('threat_actors', []),
            text=report[:200],
            confidence=confidence
        )]

    return []

def _generate_action_plan(self, mappings: List[TTMapping], report: str) -> ActionPlan:
    """Generate remediation action plan for organizations and individuals"""

    # Get mitigations for identified techniques
    org_mitigations = []
    seen_mitigations = set()

    for mapping in mappings:
        # Extract technique ID
        tech_id = mapping.technique.split(' - ')[0] if ' - ' in mapping.technique else
mapping.technique

        mitigations = self.knowledge_base.mitigation_lookup.get(tech_id, [])
        for mitigation in mitigations:
            mit_key = mitigation['id']
            if mit_key not in seen_mitigations:
                seen_mitigations.add(mit_key)
                org_mitigations.append({
                    "mitigation": mitigation['name'],
```

```
            "attack_id": mitigation['id'],
            "description": mitigation['description'][:200] + "..."
        })

    # Generate individual action checklist
    techniques_summary = "\n".join([
        f"- {m.technique}: {m.procedure}" for m in mappings[:5]
    ])

    prompt = f"""Based on this threat analysis, create an immediate action checklist for individuals who have been hacked.

IDENTIFIED TECHNIQUES:
{techniques_summary}

THREAT REPORT CONTEXT:
{report[:500]}

Create a prioritized checklist of 5-10 immediate actions an individual should take. Focus on:
1. Immediate containment (disconnect, change passwords, etc.)
2. Evidence preservation
3. System cleanup
4. Recovery steps
5. Prevention measures

Respond with a JSON array of action items in this format:
["Action 1", "Action 2", ...]

Keep actions clear, specific, and actionable. Respond ONLY with the JSON array."""

    response = self._generate_with_llm(prompt, max_tokens=1024)
```

```
    # Parse individual actions
    individual_actions = []
    try:
        json_match = re.search(r'\[.*\]', response, re.DOTALL)
        if json_match:
            individual_actions = json.loads(json_match.group())
    except json.JSONDecodeError:
        # Fallback actions
        individual_actions = [
            "Immediately disconnect the affected system from the network",
            "Change all passwords from a clean device",
            "Enable multi-factor authentication on all accounts",
            "Run a full system antivirus scan",
            "Review recent account activity for unauthorized access",
            "Back up important files to secure offline storage",
            "Contact your IT department or security team",
            "Monitor financial accounts for suspicious activity"
        ]

    return ActionPlan(
        for_organizations=org_mitigations,
        for_individuals=individual_actions
    )

def _generate_summary(self, report: str, mappings: List[TTMapping]) -> str:
    """Generate human-readable summary of what happened and how"""

    techniques_context = "\n".join([
        f"- {m.tactic} → {m.technique}: {m.procedure}"
```

```
        for m in mappings[:5]
    ])

    prompt = f"""Provide a clear, concise summary of this cybersecurity incident for a
non-technical audience.

THREAT REPORT:
{report}

IDENTIFIED TECHNIQUES:
{techniques_context}

Write a 2-3 paragraph summary that explains:
1. WHAT happened (what type of attack)
2. HOW it happened (attack methodology)
3. WHY it matters (potential impact)

Use clear language that a non-expert can understand. Avoid excessive jargon."""

    summary = self._generate_with_llm(prompt, max_tokens=512)
    return summary

  def analyze(self, report: str) -> ThreatAnalysis:
    """
    Complete analysis pipeline: map report to TTPs and generate action plan.

    Args:
        report: Threat intelligence report text

    Returns:
        ThreatAnalysis object with mappings, action plan, and summary
```

```
"""
print("\n=== Starting Threat Analysis ===\n")

# Step 1: Extract threat indicators
print("Step 1: Extracting threat indicators...")
extracted_info = self._extract_threat_indicators(report)
print(f"Found: {len(extracted_info.get('threat_actors', []))} threat actors, "
      f"{len(extracted_info.get('key_behaviors', []))} behaviors")

# Step 2: Map to TTPs
print("\nStep 2: Mapping to MITRE ATT&CK TTPs...")
mappings = self._map_to_ttps(report, extracted_info)
print(f"Identified {len(mappings)} TTP mappings")

# Step 3: Generate action plan
print("\nStep 3: Generating remediation action plan...")
action_plan = self._generate_action_plan(mappings, report)
print(f"Created action plan with {len(action_plan.for_organizations)} org mitigations "
      f"and {len(action_plan.for_individuals)} individual actions")

# Step 4: Generate summary
print("\nStep 4: Generating threat summary...")
summary = self._generate_summary(report, mappings)

print("\n=== Analysis Complete ===\n")

return ThreatAnalysis(
    mappings=mappings,
    action_plan=action_plan,
    summary=summary
```

```
    )

# Cell 5
if __name__ == "__main__":
    # Initialize
    print("Initializing MITREMapper...")
    kb = MITREKnowledgeBase()
    mapper = MITREMapper(knowledge_base=kb)

    # Example threat report
    example_report = """
        The threat actor APT29 compromised the organization's network through a
        spear-phishing email containing a malicious attachment. Once executed, the malware
        established persistence   using scheduled tasks and began lateral movement using
        stolen credentials. The attackers deployed PowerShell scripts to exfiltrate sensitive
        data to a command-and-control server. They also used WMI for remote execution on
        multiple Windows systems.
    """

    # Analyze
    analysis = mapper.analyze(example_report)

    # Output results
    print("\n" + "="*80)
    print("THREAT ANALYSIS RESULTS")
    print("="*80)

    print("\n--- SUMMARY ---")
    print(analysis.summary)

    print("\n--- TTP MAPPINGS ---")
    for i, mapping in enumerate(analysis.mappings, 1):
```

```
        print(f"\n{i}. Tactic: {mapping.tactic}")
        print(f"   Technique: {mapping.technique}")
        if mapping.sub_technique:
            print(f"   Sub-technique: {mapping.sub_technique}")
        print(f"   Procedure: {mapping.procedure}")
        print(f"   Evidence: \"{mapping.text}\"")
        print(f"   Confidence: {mapping.confidence:.2f}")
        if mapping.threat_actors:
            print(f"   Threat Actors: {', '.join(mapping.threat_actors)}")

    print("\n--- ACTION PLAN: FOR ORGANIZATIONS ---")
    for i, mitigation in enumerate(analysis.action_plan.for_organizations, 1):
        print(f"\n{i}. [{mitigation['attack_id']}] {mitigation['mitigation']}")
        print(f"   {mitigation['description']}")

    print("\n--- ACTION PLAN: FOR INDIVIDUALS ---")
    for i, action in enumerate(analysis.action_plan.for_individuals, 1):
        print(f"{i}. {action}")

    # Save results
    output_path = "threat_analysis_results.json"
    with open(output_path, 'w', encoding='utf-8') as f:
        json.dump(analysis.to_dict(), f, indent=2)
    print(f"\n\nResults saved to {output_path}")
```

Conclusion

The MITREMapper project successfully addresses the challenging and laborious task of manually mapping unstructured observed behaviors to the structured MITRE ATT&CK framework. By leveraging a modern, efficient Large Language Model (Llama 3.2), this tool automates the analysis from start to finish.

The project's key innovation lies in its multi-faceted approach. It combines:

1. Few-Shot Prompting to guide the model's output format for reliable, structured JSON results.
2. A Multi-Step Analysis that not only identifies attacker techniques but also generates practical, actionable remediation plans for both cybersecurity professionals and non-technical individuals.

MITREMapper serves as a powerful force multiplier for security analysts, reducing manual effort and enabling faster, more consistent threat analysis. Furthermore, by generating clear summaries and personalized safety checklists, it bridges the gap between complex cyber threats and the everyday user, empowering individuals to better protect themselves in an increasingly hostile digital world.